Lab U1 - "ROS 2 from Scratch" (Packt), chapter 9: Python and C++ nodes

Aim. Build and run the *unmodified* chapter-9 code of the Packt book **ROS 2 from Scratch**: publishers, a subscriber, a service, parameters, namespaces, launch files and an action - each in Python and in C++.

Time. 90 min. **Needs.** Only this folder, plus a C++ compiler (Visual Studio Build Tools, C++ workload) for the book's C++ nodes and its interface package. The book's repository is downloaded by `pixi run build-packt`. The code lives in `upstream\ros2-from-scratch\ch9` and is built into its own overlay `upstream_ws\packt`; the clone is never modified.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and Visual Studio Build Tools (C++ workload); internet access once, to download the book's repository from GitHub.

In this folder	
`upstream\`	the original repository (downloaded by the first build task, pinned commit); `upstream_ws\` its build
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

Package	Content
`my_py_pkg`, `my_cpp_pkg`	the same nodes in two languages: `number_publisher`, `number_counter` (service `reset_counter`), `count_until_server` / `count_until_client` (action)
`my_robot_interfaces`	`ResetCounter.srv` (`int64 reset_value -> bool success, string message`), `ActivateTurtle.srv`, `CountUntil.action` (goal `target_number, delay`; feedback `current_number`; result `reached_number`)
`turtle_controller` (Python), `turtle_controller_cpp`	drives a turtle in a circle, changes the pen colour, service `activate_turtle`
`my_robot_bringup`	launch files and YAML parameter files

A **namespace** (`/abc` , `/t1`) puts every name of a node under a prefix, so the same node can run twice. A **remap** renames a topic at start-up.

Procedure

Commands that need the book's own interface types go through its overlay. This one-line shortcut (re-type it in every new terminal) makes them short:

```
function packt { pixi run cmd /c scripts\in_upstream.bat packt @args }
```

1. Build the book's six packages (about 1-5 minutes the first time, seconds afterwards).

```
pixi run build-packt
```

Part A - publishers, counter, service, parameters

2. T1 - start the number app. Two Python publishers (inline parameters / YAML parameters) feed one C++ counter.

```
pixi run demo-packt-numbers
```

3. T2 - inspect. The launch file asks for a remap `/number -> /my_number` , but the topic really is `/abc/number` (see question 1).

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros topic list --no-daemon --spin-time 5
pixi run ros topic echo /abc/number --no-daemon --once --spin-time 5
```

4. T2 - parameters. `num_pub1` was configured in the launch file (period 1.5 s), `num_pub2` from the YAML file (number 4). Changing `number` at run time takes effect immediately (a callback in the node).

```
pixi run ros param get /abc/num_pub1 publish_period --no-daemon --spin-time 5
pixi run ros param get /abc/num_pub2 number --no-daemon --spin-time 5
pixi run ros param set /abc/num_pub1 number 10 --spin-time 5
```

5. T2 - call the C++ service. It must refuse a negative value and a value above the current count, and accept a valid one (watch T1: `Reset counter to 0`).

```
function packt { pixi run cmd /c scripts\in_upstream.bat packt @args }
packt ros2 service call /abc/reset_counter my_robot_interfaces/srv/ResetCounter "{reset_value: -5}"
packt ros2 service call /abc/reset_counter my_robot_interfaces/srv/ResetCounter "{reset_value:
99999999}"
packt ros2 service call /abc/reset_counter my_robot_interfaces/srv/ResetCounter "{reset_value: 0}"
```

```
# Ctrl+C in T1
```

Part B - two turtles, two namespaces

6. T1 - two turtlesim windows (red and green background), each with its own controller (`turtle_velocity` 1.5 and 0.5 from the YAML file).

```
pixi run demo-packt-turtles
```

7. T2 - compare the two controllers, then stop one.

```
function packt { pixi run cmd /c scripts\in_upstream.bat packt @args }
pixi run ros param get /t1/turtle_controller turtle_velocity --no-daemon --spin-time 5
pixi run ros param get /t2/turtle_controller turtle_velocity --no-daemon --spin-time 5
packt ros2 service call /t1/activate_turtle my_robot_interfaces/srv/ActivateTurtle "{activate:
false}"
```

Watch the windows: the left turtle stops, the other keeps circling. Confirm with the topic (no message arrives for `/t1`, messages keep coming for `/t2`; `linear.x` is 0.5, or 1.0 while the turtle is in the right half of the window, because the controller doubles the speed there):

```
pixi run ros topic echo /t2/turtle1/cmd_vel --no-daemon --once --spin-time 5
```

```
pixi run ros topic echo /t1/turtle1/cmd_vel --no-daemon --spin-time 5
```

```
# Ctrl+C in T1
```

Part C - the action `count_until`, Python and C++

8. T1 - action server (Python), T2 - client. The client sends goal "count to 5, 0.5 s apart", prints feedback 1..5 and the result. **The client keeps running after the result: press Ctrl+C.**

```
pixi run cmd /c scripts\in_upstream.bat packt ros2 run my_py_pkg count_until_server
```

```
pixi run cmd /c scripts\in_upstream.bat packt ros2 run my_py_pkg count_until_client
```

Repeat with the C++ pair (stop the Python server first): the behaviour is the same.

```
# Ctrl+C in T1 (server) and T2 (client)
```

```
pixi run cmd /c scripts\in_upstream.bat packt ros2 run my_cpp_pkg count_until_server
```

```
pixi run cmd /c scripts\in_upstream.bat packt ros2 run my_cpp_pkg count_until_client
```

```
# Ctrl+C in T1 (server) and T2 (client)
```

Observations (recorded)

Item	Value
Topic of the number app	`/abc/number` (not `/my_number`)

Item	Value
`reset_counter` replies	`-5 -> success=False, 'Cannot reset counter to a negative value'`; too large `-> success=False, 'Reset value must be lower than current counter value'`; `0 -> success=True`
`turtle_velocity`	`/t1`: 1.5, `/t2`: 0.5; `cmd_vel` linear.x 1.5 and 0.5
`activate_turtle(false)` on `/t1`	reply `Stopping the turtle`; `/t1/turtle1/cmd_vel` falls silent, `/t2` unaffected
`count_until`	feedback 1,2,3,4,5 at 0.5 s steps, result 5 - identical in Python and C++

Verification

```
pixi run test-packt
```

25 checks against the book's code: C++ service (negative / too high / valid), YAML and inline launch parameters, the fast turtle travels about 22 units in 10 s against 8-9 for the slow one, `activate_turtle`, and the `count_until` action in both languages (accept, 5 feedback messages, result, reject a negative target, cancel a running goal).

Questions

1. In `number_app.launch.py` the remap is `("/number", "/my_number")`, yet the topic is `/abc/number`. Why does the rule not match, and how would you write a remap that works?
2. What does `ros2 param set /abc/num_pub1 number 10` change in the counter's total, and why does it work while the node is running?
3. Why does the service refuse `reset_value` above the current count? Where in `number_counter.cpp` is that decided?
4. What is the advantage of an action over a service for `count_until`? What would `Ctrl+C` in the client do to a *running* goal?
5. *(Extension)* Start the second Python publisher with another namespace and see that the counter is not affected.

If it does not work

Symptom	Fix
`in_upstream.bat: upstream_ws\packt is not built`	`pixi run build-packt`
Service call hangs on "waiting for service"	T1 is not running, or the namespace is wrong (`/abc/reset_counter`)
Client sees no server right after you restarted the server	wait 5 s; ROS still remembers the old server for a moment